



INSTITUTO POLITÉCNICO DE BEJA
Escola Superior de Tecnologia e Gestão
Mestrado em Engenharia de Segurança Informática
Linguagens de Programação Dinâmicas

Trabalho de Python

Paulo António Tavares Abade - 23919



Beja, fevereiro de 2026

INSTITUTO POLITÉCNICO DE BEJA
Escola Superior de Tecnologia e Gestão
Mestrado em Engenharia de Segurança Informática
Linguagens de Programação Dinâmicas

Trabalho de Python

Paulo António Tavares Abade - 23919

Orientador: Armando de Jesus Ventura

Beja, fevereiro de 2026

Resumo

Neste trabalho será apresentada a resolução do trabalho prático referente à parte Python, onde é pedida a criação de uma aplicação com menu em linha de comandos que permita ao utilizador: Fazer Port Scanning, fazer um DoS Attack a um endereço IPv4, fazer um Syn Flood Attack a um endereço IPv4, fazer um script que analise ficheiros de log de pelo menos dois serviços, fazer um script de troca de mensagens encriptadas entre um cliente e um servidor, fazer um cliente de Port Knocking e fazer um script de gestão de passwords. Tudo isto utilizando a linguagem Python e algumas das bibliotecas disponíveis para a mesma, em um ambiente virtual com o sistema operativo Kali Linux.

Keywords: PortScanning, DoS Attack, Syn Flood Attack, Análise de Logs, Mensagens Encriptadas, Port Knocking, Gestão de Passwords

Abstract

This work will present the solution to the practical assignment related to the Python portion, which requires the creation of an application with a command-line menu that allows the user to: Perform Port Scanning, perform a DoS attack on an IPv4 address, perform a Syn Flood attack on an IPv4 address, create a script that analyzes log files from at least two services, create a script for exchanging encrypted messages between a client and a server, create a Port Knocking client, and create a password management script. All of this using the Python language and some of the available libraries, in a virtual environment with the Kali Linux operating system.

Keywords: PortScanning, DoS Attack, Syn Flood Attack, Log Analysis, Encrypted Messages, Port Knocking, Password Management

Índice

1	Introdução	1
1.1	Análise dos Requisitos	2
1.2	Explicação Teórica de alguns temas	3
1.2.1	Port Scanning	3
1.2.2	DoS Attack	4
1.2.3	Syn Flood Attack	4
1.2.4	Port Knocking	5
2	Desenvolvimento	6
2.1	Menu	6
2.2	Port Scanner	7
2.3	DoS Attack	8
2.4	Syn Flood Attack	8
2.5	Analisador de Logs	9
2.6	Port Knocker	11
2.7	Gestor de Passwords	12
2.8	Servidor de Mensagens	13
2.9	Ficheiros auxiliares	14
3	Conclusão	15
	Bibliografia	16

Índice de Figuras

1	Exemplo do menu em linha de comandos, onde o utilizador pode escolher qual funcionalidade deseja utilizar	1
2	O Nmap é uma ferramenta popular para realizar Port Scanning	3
3	Exemplo de um DoS Attack, onde o atacante envia uma grande quantidade de tráfego para o alvo, sobrecarregando o sistema alvo	4

4	Exemplo de um Syn Flood Attack, onde o atacante envia uma grande quantidade de pacotes SYN para o alvo, sem completar o handshake, sobrecarregando o sistema alvo	4
5	Exemplo de Port Knocking, onde o cliente envia uma sequência específica de pacotes para portas fechadas em um sistema, e quando a sequência correta é recebida, o sistema adiciona o endereço IP do remetente a uma lista de permissões temporária, permitindo o acesso ao dispositivo	5
6	Exemplo do Menu em Linha de Comandos	6
7	Exemplo do resultado do Port Scanning, mostrando quais portas estão abertas, fechadas ou filtradas	7
8	Exemplo do resultado do Port Scanning, mostrando apenas as portas que estão abertas	7
9	Exemplo de funcionamento do Log Analyzer	9
10	Exemplo do relatório em pdf gerado pelo Log Analyzer	10
11	Exemplo do relatório em csv gerado pelo Log Analyzer	10
12	Exemplo do funcionamento do Port Knocking, onde o cliente envia uma sequência específica de pacotes para portas fechadas em um sistema, e quando a sequência correta é recebida, o sistema adiciona o endereço IP do remetente a uma lista de permissões temporária, permitindo o acesso ao dispositivo	11
13	Exemplo do resultado do Port Knocking, onde o sistema alvo adiciona o endereço IP do cliente à lista de permissões temporária, permitindo o acesso ao dispositivo	11
14	Exemplo de funcionamento do gestor de passwords e os ficheiros que são gerados	12
15	Exemplo de funcionamento do servidor e do cliente de mensagens	13

1 Introdução

Neste trabalho será apresentada a resolução do trabalho prático referente à parte Python, onde é pedida a criação de uma aplicação com menu em linha de comandos que permita ao utilizador: Fazer Port Scanning, fazer um DoS Attack a um endereço IPv4, fazer um Syn Flood Attack a um endereço IPv4, fazer um script que analise ficheiros de log de pelo menos dois serviços, fazer um script de troca de mensagens encriptadas entre um cliente e um servidor, fazer um cliente de Port Knocking e fazer um script de gestão de passwords. Tudo isto foi realizado dentro de uma máquina virtual com o sistema operativo Kali Linux, utilizando a linguagem Python e algumas das bibliotecas disponíveis para a mesma. Foi testado em um ambiente controlado, onde havia permissão para realizar os ataques e análises necessárias. Todo o desenvolvimento do projeto pode ser encontrado no repositório do GitHub (Abade, 2025), junto de todos os commits feitos e não apenas os últimos como é possível ver na imagem Abade, 2025.

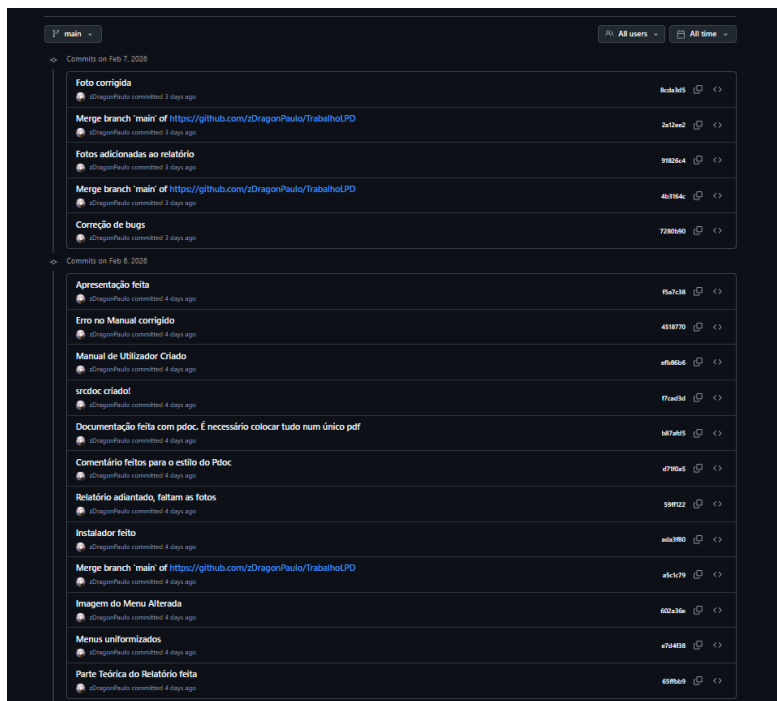


Figura 1: Exemplo do menu em linha de comandos, onde o utilizador pode escolher qual funcionalidade deseja utilizar

1.1 Análise dos Requisitos

Para a realização deste trabalho prático, foi primeiro necessário compreender tudo o que estava a ser pedido para realizar o trabalho, sendo analisado nas próximas secções os temas mais teóricos que necessitaram de ser aprofundados para ser possível a sua implementação.

Sendo assim, o que será feito neste trabalho é:

- Implementar um menu em linha de comandos para permitir ao utilizador escolher qual funcionalidade deseja utilizar.
- Implementar a funcionalidade de Port Scanning, vendo quais portas estão abertas, fechadas ou filtradas em um sistema de rede.
- Implementar a funcionalidade de DoS Attack, enviando uma grande quantidade de tráfego para um alvo, através do protocolo UDP, para sobrecarregar o sistema alvo.
- Implementar a funcionalidade de Syn Flood Attack, enviando uma grande quantidade de pacotes SYN para um alvo, sem completar o handshake, para sobrecarregar o router do sistema alvo.
- Implementar a funcionalidade de análise de logs, para analisar ficheiros de log do ssh e do http, e serão criados relatórios em pdf e csv para estes. O relatório irá conter informações sobre a data e hora dos eventos, o endereço IP de origem, o tipo de evento e uma breve descrição do evento e caso seja possível, o país e cidade de origem do endereço IP. Para isso, serão utilizadas as bibliotecas *geolite2* para obter a localização geográfica dos endereços IP, *FPDF* para criar os relatórios em pdf e *csv* para criar os relatórios em csv.
- Implementar a funcionalidade de troca de mensagens encriptadas entre um cliente e um servidor, utilizando o protocolo TCP para a comunicação entre ambos, e utilizando a biblioteca *cryptography* do Python para a encriptação e desencriptação das mensagens. O objetivo será para um arquivamento seguro das mensagens, onde apenas o cliente e o servidor terão acesso ao conteúdo das mensagens.

- Implementar a funcionalidade de Port Knocking, onde o cliente irá enviar uma sequência específica de pacotes para portas fechadas em um sistema, e quando a sequência correta for recebida, o sistema irá adicionar o endereço IP do remetente a uma lista de permissões temporária, permitindo o acesso ao dispositivo.
- Implementar a funcionalidade de gestão de passwords, onde o utilizador poderá criar, ler, atualizar e eliminar passwords de forma segura, utilizando a biblioteca *cryptography* do Python para encriptar as passwords antes de as armazenar em uma base de dados SQLite, e para descriptar as passwords quando o utilizador quiser visualizá-las com recurso a 2FA. manualmente.

1.2 Explicação Teórica de alguns temas

1.2.1 Port Scanning

O Port Scanning é uma técnica utilizada para identificar quais portas estão abertas, a escutar ou fechadas em um sistema de rede. Um exemplo de ferramenta popular para realizar Port Scanning é o Nmap (N. Team, 2025), como mostrado na Figura 2. O Nmap pode ser utilizado para descobrir hosts e serviços em uma rede, criando um "mapa" da rede.

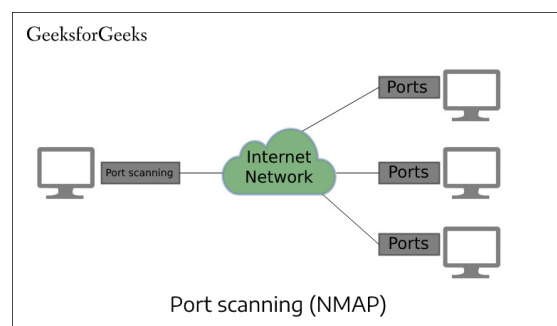


Figura 2: O Nmap é uma ferramenta popular para realizar Port Scanning

1.2.2 DoS Attack

Um DoS (Denial of Service) Attack (Wikipedia, 2025a) é um ataque que tem o objetivo de tornar um serviço indisponível para um ou mais utilizadores. Isso é feito através do envio de uma grande quantidade de tráfego para o alvo, através do protocolo UDP, já que este não espera uma resposta, o que facilita a sobrecarga do sistema alvo.

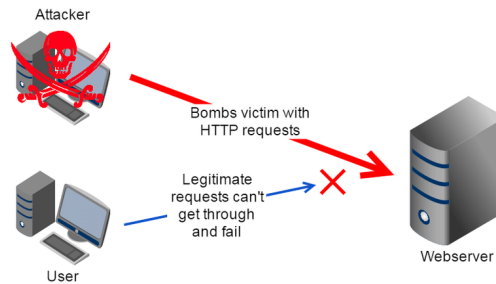


Figura 3: Exemplo de um DoS Attack, onde o atacante envia uma grande quantidade de tráfego para o alvo, sobrecarregando o sistema alvo

1.2.3 Syn Flood Attack

Um Syn Flood Attack (Wikipedia, 2025c) é um tipo de ataque que explora o processo de handshake do protocolo TCP. O atacante envia uma grande quantidade de pacotes SYN para o alvo, mas nunca completa o handshake, o que leva o sistema a ficar sobrecarregado com conexões half-open. Isto pode resultar na indisponibilidade do serviço para os utilizadores.

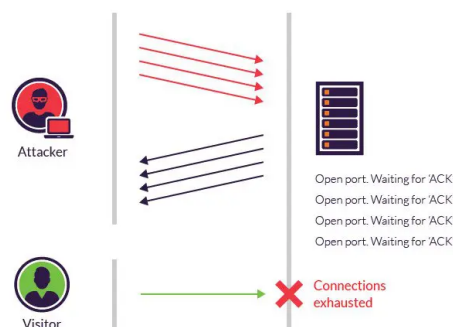


Figura 4: Exemplo de um Syn Flood Attack, onde o atacante envia uma grande quantidade de pacotes SYN para o alvo, sem completar o handshake, sobrecarregando o sistema alvo

1.2.4 Port Knocking

O Port Knocking Wikipedia, 2025b é uma técnica de segurança que envolve o envio de uma sequência específica de pacotes para portas fechadas em um sistema. Quando a sequência correta é recebida, o sistema adiciona o endereço IP do remetente a uma lista de permissões temporária, permitindo o acesso ao dispositivo.

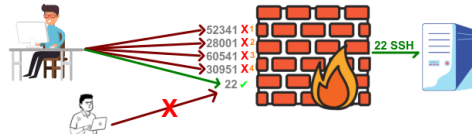


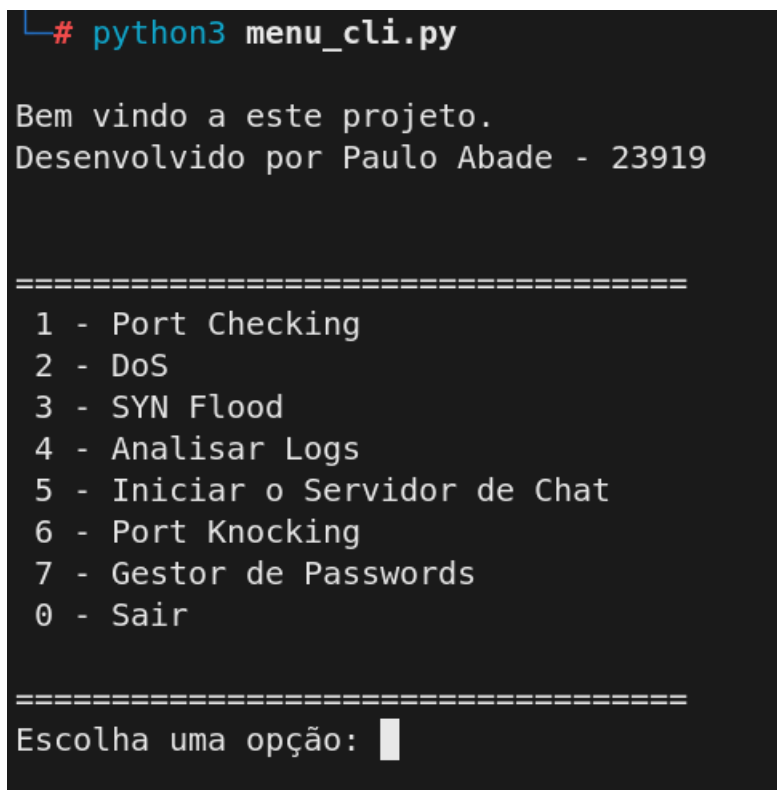
Figura 5: Exemplo de Port Knocking, onde o cliente envia uma sequência específica de pacotes para portas fechadas em um sistema, e quando a sequência correta é recebida, o sistema adiciona o endereço IP do remetente a uma lista de permissões temporária, permitindo o acesso ao dispositivo

2 Desenvolvimento

Nesta secção será apresentado o desenvolvimento do trabalho prático, com a explicação de cada uma das funcionalidades implementadas.

2.1 Menu

Foi criado um simples menu em linha de comandos para permitir ao utilizador escolher qual funcionalidade deseja utilizar. O menu é apresentado em um loop, permitindo ao utilizador escolher várias funcionalidades sem precisar reiniciar o programa. O menu é implementado utilizando a função de *match case* do Python, que é equivalente a um switch case em outras linguagens de programação, e é uma forma simples e eficiente de lidar com múltiplas opções de escolha.



```
# python3 menu_cli.py

Bem vindo a este projeto.
Desenvolvido por Paulo Abade - 23919

=====
1 - Port Checking
2 - DoS
3 - SYN Flood
4 - Analisar Logs
5 - Iniciar o Servidor de Chat
6 - Port Knocking
7 - Gestor de Passwords
0 - Sair
=====

Escolha uma opção: █
```

Figura 6: Exemplo do Menu em Linha de Comandos

2.2 Port Scanner

A funcionalidade de Port Scanning foi implementada utilizando a biblioteca *nmap* do Python. Esta biblioteca permite a realização de varreduras de portas de forma simples e eficiente. O código pede ao utilizador um endereço IPv4, e em seguida pede um intervalo de portas a serem escaneadas. O resultado do escaneamento é apresentado ao utilizador, mostrando quais portas estão abertas, fechadas ou filtradas.

```
--- Reconhecimento de Rede: Port Scanner ---  
Insira um IP dentro da sua rede privada (ex: 192.168.1.254):  
192.168.1.254  
Insira a porta inicial (1-65535):  
20  
Insira a porta final (20-65535):  
30  
[*] A realizar scan em 192.168.1.254 (Portas: 20-30)...  
  
Resultados para o host: 192.168.1.254  
-----  
Porta    20 | Estado: closed  
Porta    21 | Estado: open  
Porta    22 | Estado: open  
Porta    23 | Estado: open  
Porta    24 | Estado: closed  
Porta    25 | Estado: closed  
Porta    26 | Estado: closed  
Porta    27 | Estado: closed  
Porta    28 | Estado: closed  
Porta    29 | Estado: closed  
Porta    30 | Estado: closed
```

Figura 7: Exemplo do resultado do Port Scanning, mostrando quais portas estão abertas, fechadas ou filtradas

No fim, ainda é apresentado apenas as portas que há certeza que estão abertas, para facilitar a leitura, como se pode ver na figura 8.

```
-----  
Estão abertas as portas:  
Porta    21 | Estado: open  
Porta    22 | Estado: open  
Porta    23 | Estado: open  
-----
```

Figura 8: Exemplo do resultado do Port Scanning, mostrando apenas as portas que estão abertas

2.3 DoS Attack

A funcionalidade de DoS Attack foi implementada utilizando a biblioteca ***Sockets*** do Python. Esta biblioteca permite a criação e envio de pacotes de rede de forma flexível. O código pede ao utilizador um endereço IPv4 alvo, e em seguida pede a porta onde será feito o ataque. O código então envia uma grande quantidade de pacotes UDP para o alvo, com o objetivo de sobrecarregar o sistema. O código foi feito com base no exemplo feito pelo professor Armando Ventura na aula, e foi adotado por ser simples e ter sido previamente testado em um ambiente controlado, sendo que apenas foram feitos aperfeiçoamentos para melhorar a resistência do código a erros e para permitir o envio de pacotes de forma mais rápida, utilizando threads. ***random***.

2.4 Syn Flood Attack

A funcionalidade de Syn Flood Attack também foi implementada utilizando a biblioteca ***scapy*** S. Team, 2025 do Python. O código pede ao utilizador um endereço IPv4 alvo, e em seguida pede a porta onde será feito o ataque. O código então envia uma grande quantidade de pacotes SYN para o alvo, sem completar o handshake, com o objetivo de sobrecarregar o sistema. O código foi feito originalmente a partir do exemplo apresentado no seguinte GitHub <https://thepythoncode.com/article/syn-flooding-attack-using-scapy-in-python>. São utilizadas threads para enviar os pacotes de forma mais rápida. As bibliotecas utilizadas são: ***scapy*** e ***random***.

2.5 Analisador de Logs

A funcionalidade de análise de logs foi implementada utilizando as bibliotecas *geolite2*, *FPDF* e *csv* do Python. O código vai ao caminho conhecido do ficheiro de logs do sistema, e em seguida analisa o conteúdo do ficheiro, extraindo informações relevantes como a data e hora dos eventos, o endereço IP de origem, o tipo de evento e uma breve descrição do evento. O código também utiliza a biblioteca *geolite2* MaxMind, 2025 para obter a localização geográfica dos endereços IP, e cria relatórios em pdf e csv com as informações extraídas. Para funcionar, foi necessário instalar ainda o *rsyslog* no Kali Linux, já que em algumas situações, a instalação por defeito do rsyslog foi abandonada pela maioria das distribuições Linux, sendo necessário instalar o rsyslog para que os logs do sistema para o ssh sejam gerados e possam ser analisados pelo código. Os do http já são gerados por defeito, e não foi necessário instalar nenhum serviço adicional para que estes sejam gerados. Para testar a funcionalidade dos logs, foram realizadas algumas tentativas de conexão, mas como estava a ser testado por uma máquina em rede interna, não foi possível obter a localização geográfica dos endereços IP, já que estes eram endereços IP privados, e a base de dados do *geolite2* não tem informações sobre endereços IP privados. Sendo assim, foi "falsificado" o endereço IP de origem dos eventos para um endereço IP público, para que fosse possível obter a localização geográfica dos mesmos e testar a funcionalidade do código.

```
--- Analisador de Logs (SSH & HTTP) ---  
  
[+] Analisando SSH: /var/log/auth.log  
[2026-01-24 19:10:25] SSH Fail: 192.168.1.76 (Interno/Inválido, N/A)  
[2026-01-24 19:21:42] SSH Fail: 8.8.8.8 (United States, Desconhecida)  
[2026-01-24 19:25:43] SSH Fail: 81.92.203.210 (United Kingdom, Poplar)  
  
[+] Analisando HTTP: /var/log/apache2/access.log  
[2026-01-24 19:17:02] HTTP 404: 192.168.1.76 (Interno/Inválido, N/A)  
[2026-01-24 19:17:02] HTTP 404: 192.168.1.76 (Interno/Inválido, N/A)  
[2026-01-24 19:22:24] HTTP 404: 1.2.3.4 (Australia, Desconhecida)  
[2026-01-24 19:25:58] HTTP 404: 212.58.244.70 (United Kingdom, Desconhecida)  
  
Deseja gerar relatório em PDF e CSV? (s/n): s  
  
[+] PDF gerado com sucesso: Relatorio_Logs.pdf  
[+] CSV gerado com sucesso: relatorio_ataques.csv
```

Figura 9: Exemplo de funcionamento do Log Analyzer

Em seguida, é possível ver na figura 10 o exemplo do relatório em pdf gerado pelo código, onde é possível ver as informações extraídas dos logs, incluindo a localização geográfica dos endereços IP.

Relatório de Segurança - Análise de Logs

Gerado em: 07/02/2026 16:16:56

Timestamp	IP	País	Cidade	Detalhes
2026-01-24 19:10:25	192.168.1.76	Interno/Inválido	N/A	SSH Login Failed
2026-01-24 19:17:02	192.168.1.76	Interno/Inválido	N/A	HTTP Error 404
2026-01-24 19:17:02	192.168.1.76	Interno/Inválido	N/A	HTTP Error 404
2026-01-24 19:21:42	8.8.8.8	United States	Desconhecida	SSH Login Failed
2026-01-24 19:22:24	1.2.3.4	Australia	Desconhecida	HTTP Error 404
2026-01-24 19:25:43	81.92.203.210	United Kingdom	Poplar	SSH Login Failed
2026-01-24 19:25:58	212.58.244.70	United Kingdom	Desconhecida	HTTP Error 404

Figura 10: Exemplo do relatório em pdf gerado pelo Log Analyzer

É possível também ver na figura 11 o exemplo do relatório em csv gerado pelo código, onde é possível ver as informações extraídas dos logs, incluindo a localização geográfica dos endereços IP, de forma mais estruturada e fácil de ler. desta maneira pode ser inserido mais facilmente em bases de dados e/ou excel's.

```
relatorio_ataques.csv
Timestamp;IP;País;Cidade;Evento
2026-01-24 19:10:25;192.168.1.76;Interno/Inválido;N/A;SSH Login Failed
2026-01-24 19:17:02;192.168.1.76;Interno/Inválido;N/A;HTTP Error 404
2026-01-24 19:17:02;192.168.1.76;Interno/Inválido;N/A;HTTP Error 404
2026-01-24 19:21:42;8.8.8.8;United States;Desconhecida;SSH Login Failed
2026-01-24 19:22:24;1.2.3.4;Australia;Desconhecida;HTTP Error 404
2026-01-24 19:25:43;81.92.203.210;United Kingdom;Poplar;SSH Login Failed
2026-01-24 19:25:58;212.58.244.70;United Kingdom;Desconhecida;HTTP Error 404
```

Figura 11: Exemplo do relatório em csv gerado pelo Log Analyzer

2.6 Port Knocker

O Port Knocking foi implementado utilizando a biblioteca *scapy* do Python para enviar os pacotes de forma personalizada, onde basicamente o utilizador escolhe um endereço IPv4 alvo, informa o número de portas, e depois informa a sequência de portas para o Port Knocking. O código então envia pacotes TCP para as portas especificadas, com o objetivo de "bater" na porta e permitir o acesso ao dispositivo alvo. Para testar esta funcionalidade, foi utilizado um servidor Alma Linux 8.10, já que este sistema operativo já tinha sido configurado para esta funcionalidade em um trabalho prático anterior, sendo assim mais fácil de reutilizar a configuração já existente para esse sistema operativo.

```
===== Port Knocking Client =====  
  
Insira o IP do servidor (ex: 192.168.1.86):  
192.168.1.87  
Quantas portas tem a sequência de knock? 3  
Insira as 3 portas pela ordem correta:  
Porta 1: 4444  
Porta 2: 3333  
Porta 3: 2222  
  
[*] A iniciar sequência de knock para 192.168.1.87...  
[+] Knock enviado para a porta: 4444  
[+] Knock enviado para a porta: 3333  
[+] Knock enviado para a porta: 2222  
[!] Sequência completada. Tente aceder ao serviço agora.
```

Figura 12: Exemplo do funcionamento do Port Knocking, onde o cliente envia uma sequência específica de pacotes para portas fechadas em um sistema, e quando a sequência correta é recebida, o sistema adiciona o endereço IP do remetente a uma lista de permissões temporária, permitindo o acesso ao dispositivo

E como resultado o Alma Linux 8.10 adiciona o endereço IP do cliente à lista de permissões temporária, permitindo o acesso ao dispositivo, como se pode ver na figura 13.

```
[2026-02-07 16:26] starting up, listening on emp0s3  
[2026-02-07 16:26] 192.168.1.86: openSSH: Stage 1  
[2026-02-07 16:26] 192.168.1.86: openSSH: Stage 2  
[2026-02-07 16:26] 192.168.1.86: openSSH: Stage 3  
[2026-02-07 16:26] 192.168.1.86: openSSH: OPEN SESAME  
[2026-02-07 16:26] openSSH: running command: /sbin/iptables -I INPUT -s 192.168.1.86 -p tcp --dport ssh -j ACCEPT
```

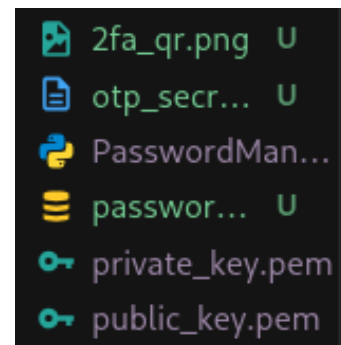
Figura 13: Exemplo do resultado do Port Knocking, onde o sistema alvo adiciona o endereço IP do cliente à lista de permissões temporária, permitindo o acesso ao dispositivo

2.7 Gestor de Passwords

A funcionalidade de gestão de passwords foi implementada utilizando a biblioteca *cryptography* do Python para encriptar as passwords antes de as armazenar em uma base de dados SQLite, e para descriptar as passwords quando o utilizador quiser visualizá-las. O código permite ao utilizador criar, ler, atualizar e eliminar registos de credenciais de forma segura, sendo que para fazer qualquer uma das últimas três ações, o utilizador tem que passar por um processo de autenticação de dois fatores (2FA) (Wikipedia, 2025d), onde é necessário inserir o código que é gerado pela aplicação de autenticação, neste caso foi utilizado o Microsoft Authenticator, mas poderia ser utilizado qualquer outra aplicação de autenticação que suporte o protocolo TOTP (Time-based One-Time Password). Para facilitar o processo de configuração do 2FA, o código gera um QR code que pode ser escaneado pela aplicação de autenticação para adicionar a conta.

```
=====
GESTOR SEGURO DE PASSWORDS
=====
1. Criar Registo
2. Listar Registos (Requer 2FA)
3. Atualizar Registo (Requer 2FA)
4. Eliminar Registo (Requer 2FA)
0. Sair
```

(a) Exemplo de funcionamento



(b) Ficheiros gerados pelo gestor de passwords

Figura 14: Exemplo de funcionamento do gestor de passwords e os ficheiros que são gerados

2.8 Servidor de Mensagens

Este foi o último módulo a ser implementado, e foi o mais complexo de todos, já que envolveu a criação de um servidor e um cliente para a troca de mensagens encriptadas. O responsável pelo servidor é o Kali Linux, ou seja, aquele deixara armazenada a base de dados com as mensagens, e o cliente é um script Python que pode ser executado em qualquer máquina, e que se conecta ao servidor para enviar e receber mensagens. O código do servidor utiliza a biblioteca *socket* para criar um servidor TCP, e a biblioteca *cryptography* para encriptar e desencriptar as mensagens. O código do cliente também utiliza a biblioteca *socket* para se conectar ao servidor, e a biblioteca *cryptography* para encriptar as mensagens antes de as enviar, e para desencriptar as mensagens recebidas do servidor.

Estes scripts de mensagens baseiam-se nas chaves assimétricas RSA para a encriptação e desencriptação das mensagens, onde o servidor gera um par de chaves (uma chave pública e uma chave privada), e o cliente utiliza a chave pública do servidor para encriptar as mensagens antes de as enviar, e o servidor utiliza a sua chave privada para desencriptar as mensagens recebidas. Para testar esta funcionalidade, foi necessário configurar o servidor para aceitar conexões na porta 9999, e depois executar o cliente em outra máquina para se conectar ao servidor e enviar mensagens. Assim que é estabelecida a conexão, existe a troca de chaves públicas entre o cliente e o servidor, para que o cliente possa encriptar as mensagens antes de as enviar, e o servidor possa desencriptar as mensagens recebidas. O cliente pode enviar mensagens, listar e fazer download das mesmas e eliminar mensagens, e o servidor irá armazenar as mensagens em ficheiros binários encriptados.

```
=====
CLIENTE DE MENSAGENS
=====
1. Enviar Mensagem Segura
2. Download e Visualizar Mensagens
3. Exportar Backup (Simétrico AES)
4. Eliminar as minhas Mensagens do Servidor
0. Sair

Escolha uma opção: █
```

(a) Cliente

```
[*] A iniciar Servidor de Mensagens Seguras...
[+] Chaves do Servidor geradas com sucesso.

[+] Servidor 'Re-Encryption' ativo na porta 9999...
```

(b) Servidor

Figura 15: Exemplo de funcionamento do servidor e do cliente de mensagens

2.9 Ficheiros auxiliares

Para instalar as dependências necessárias para o funcionamento do projeto, foi criado um script de instalação em bash, que pode ser executado na maioria dos sistemas Linux, desde que utilizem o gerenciador de pacotes apt, como é o caso do Kali Linux. O script atualiza a lista de pacotes do sistema, garante que o Python3 e o PIP estão instalados, e instala as bibliotecas necessárias para o funcionamento do projeto, como o rsyslog para a geração dos logs do ssh, o apache2 para a geração dos logs do http, e as bibliotecas do Python necessárias para as funcionalidades implementadas.

```
#!/bin/bash

echo "--- A instalar dependências para o Projeto ---"

# Atualizar a lista de pacotes do sistema
sudo apt update

# Garantir que o Python3 e o PIP estão instalados
sudo apt install -y python3 python3-pip rsyslog apache2
sudo systemctl start --now rsyslog apache2

# Instalar as bibliotecas do requirements.txt
# O --break-system-packages é necessário em versões novas do Kali/Debian
pip3 install -r requirements.txt --break-system-packages

echo "--- Instalação concluída com sucesso! ---"
```

O txt do script de instalação pode ser encontrado no ficheiro **installer.sh** na pasta **src** do projeto, junto do ficheiro **requirements.txt** que contém as bibliotecas do Python necessárias para o funcionamento do projeto.

Para além disso, foi criado um manual de instruções para explicar de forma simples o que cada funcionalidade faz, e como utilizá-las, para facilitar a utilização do projeto por parte dos utilizadores, este pode ser encontrado no ficheiro **manual.pdf**, no entanto caso seja necessário e o utilizador queira fazer uma leitura mais detalhada, pode utilizar a documentação do código, que foi feita com recurso a Pdoc (P. Team, 2025)

3 Conclusão

Foi um trabalho prático bastante complexo, que envolveu a implementação de várias funcionalidades relacionadas com a segurança informática, sendo bastante trabalhoso e exigente, especialmente na parte de desenvolvimento do servidor de mensagens, que foi o módulo mais complexo de todos. No entanto, foi um trabalho bastante interessante de ver a aplicação crescer ao longo do tempo, e de ver as funcionalidades a serem implementadas e testadas em um ambiente controlado. Foi possível aprender bastante sobre a utilização de várias bibliotecas do Python, e sobre a implementação de funcionalidades relacionadas com a segurança informática, como o DoS Attack e o Syn Flood Attack.

Bibliografia

- Abade, P. (2025). *TrabalhoLPD* [Repositório do Trabalho LPD]. Obtido fevereiro 7, 2026, de <https://github.com/zDragonPaulo/TrabalhoLPD>
- MaxMind. (2025). *GeoLite2 Free Geolocation Data* [Base de dados de geolocalização gratuita]. Obtido fevereiro 7, 2026, de <https://dev.maxmind.com/geoip/geolite2-free-geolocation-data>
- Team, N. (2025). *Nmap: Network Mapper* [Ferramenta de varredura de rede]. Obtido fevereiro 7, 2026, de <https://nmap.org/>
- Team, P. (2025). *Pdoc: Python Documentation Generator* [Gerador de documentação para Python]. Obtido fevereiro 10, 2026, de <https://pdoc.dev/>
- Team, S. (2025). *Scapy: Packet Manipulation Tool* [Ferramenta de manipulação de pacotes em Python]. Obtido fevereiro 7, 2026, de <https://scapy.net/>
- Wikipedia. (2025a). *Denial-of-Service Attack* [Ataque de negação de serviço - Wikipedia]. https://en.wikipedia.org/wiki/Denial-of-service_attack
- Wikipedia. (2025b). *Port Knocking* [Port Knocking - Wikipedia]. https://en.wikipedia.org/wiki/Port_knocking
- Wikipedia. (2025c). *SYN Flood* [SYN Flood - Wikipedia]. https://en.wikipedia.org/wiki/SYN_flood
- Wikipedia. (2025d). *Two-Factor Authentication* [Autenticação de dois fatores - Wikipedia]. https://en.wikipedia.org/wiki/Two-factor_authentication